



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

# **SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences  
Chennai-602105**

## **Autoencoder-Based Deep Learning Framework for Network Intrusion and Anomaly Detection**

### **A CAPSTONE PROJECT REPORT**

*Submitted in the partial fulfilment for the Course of*

### **MLA0408-DEEP LEARNING**

*to the award of the degree of*

### **BACHELOR OF TECHNOLOGY**

**IN**

### **ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

**Submitted by**

**HARESH KUMAR N L (192425009)**

**KUMARAN M (192525022)**

**Under the Supervision of**

**Dr.D.SUDHAGAR**



# **SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**  
**Chennai-602105**



## **DECLARATION**

We, **HARESH KUMAR N L & KUMARAN M** of the **Artificial Intelligence and Machine Learning**, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled “**Autoencoder-Based Deep Learning Framework for Network Intrusion and Anomaly Detection**” is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 08/09/2026

Signature of the Students with Names

**HARESH KUMAR N L (192425009)**

**KUMARAN M (192525022)**



**SIMATS ENGINEERING**  
**Saveetha Institute of Medical and Technical Sciences**  
**Chennai-602105**



**BONAFIDE CERTIFICATE**

This is to certify that the Capstone Project entitled “**Autoencoder-Based Deep Learning Framework for Network Intrusion and Anomaly Detection**” has been carried out by **HARESH KUMAR N L & KUMARAN M** under the supervision of Dr.D.SUDHAGAR and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech **Artificial Intelligence and Machine Learning** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE

**Dr.M. Gunasekaran**

**Program Director**

Department Name (AI & ML)

Saveetha School of Engineering

SIMATS

SIGNATURE

**Dr.D. SUDHAGAR**

**Professor**

Department of Quantum Intelligence

Saveetha School of Engineering

SIMATS

Submitted for the Project work Viva-Voce held on 08/09/2026

INTERNAL EXAMINER

EXTERNAL EXAMINER

# ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, Dr.M. Gunasekaran for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, Dr.D. Sudhagar for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name  
Haresh Kumar N L (192425009)  
Kumran M (192525022)

## Abstract

Network security has become increasingly important due to the rapid growth of cyberattacks, unauthorized access, malware, denial-of-service attacks, and abnormal network activities. Traditional intrusion detection systems mainly depend on predefined rules and signatures, which may not effectively identify new or unknown attacks. This project proposes an **Autoencoder-Based Deep Learning Framework** for detecting network intrusions and anomalies.

The proposed system uses an autoencoder to learn the normal patterns of network traffic. The network data is first preprocessed by handling missing values, encoding categorical features, scaling numerical features, and selecting relevant features. The encoder compresses the input data into a lower-dimensional latent representation, while the decoder reconstructs the original input. The difference between the original and reconstructed data is measured using **reconstruction error**. A suitable threshold is then applied to distinguish normal traffic from anomalous or potentially malicious traffic.

The framework provides an automated approach for anomaly detection and can learn complex patterns from network traffic. Performance can be evaluated using accuracy, precision, recall, F1-score, confusion matrix, and reconstruction error. The proposed approach demonstrates how deep learning can improve the detection of previously unseen network anomalies.

## Table Of Contents

| Chapter No | Title                               | Page No |
|------------|-------------------------------------|---------|
|            | Abstract                            | 5       |
| 1          | Introduction                        | 1       |
| 1.1        | Background Information              | 1       |
| 1.2        | Project Objectives                  | 1       |
| 1.3        | Significance                        | 1       |
| 1.4        | Scope                               | 2       |
| 1.5        | Methodology Overview                | 2       |
| 2          | Problem Identification and Analysis | 3       |
| 2.1        | Description of the Problem          | 3       |
| 2.2        | Evidence of the Problem             | 3       |
| 2.3        | Stakeholders                        | 3       |
| 2.4        | Supporting Data/Research            | 4       |
| 3          | Solution Design and Implementation  | 5       |
| 3.1        | Development and Design Process      | 5       |
| 3.2        | Tools and Technologies Used         | 5       |
| 3.3        | Solution Overview                   | 6       |
| 3.4        | Engineering Standards Applied       | 6       |
| 3.5        | Solution Justification              | 7       |
| 4          | Results and Recommendations         | 8       |
| 4.1        | Evaluation of Results               | 8       |
| 4.2        | Challenges Encountered              | 9       |
| 4.3        | Possible Improvements               | 10      |

|     |                                                 |    |
|-----|-------------------------------------------------|----|
| 4.4 | Recommendations                                 | 10 |
| 5   | Reflection on Learning and Personal Development | 11 |
| 5.1 | Key Learning Outcomes                           | 11 |
| 5.2 | Challenges Encountered and Overcome             | 11 |
| 5.3 | Application of Engineering Standards            | 11 |
| 5.4 | Insights into the Industry                      | 11 |
| 5.5 | Conclusion of Personal Development              | 12 |
| 6   | Conclusion                                      | 13 |
| 7   | References                                      | 14 |
| 8   | Appendices                                      | 15 |
| 8.1 | User Manual                                     | 15 |
| 8.2 | Code snippets                                   | 15 |
| 8.3 | Raw Dataset                                     | 16 |

## List of Tables

| Table/Figure No | Name                        | Page No |
|-----------------|-----------------------------|---------|
| Table 1         | Tools and Technologies Used | 5       |
| Figure 1        | DashBoard                   | 8       |
| Figure 2        | Alert Log                   | 9       |
| Figure 3        | Data Report                 | 9       |



# Chapter 1 – Introduction

## 1.1 Background Information

The increasing use of computer networks, cloud services, IoT devices, and online applications has resulted in a significant increase in cybersecurity threats. Network attacks can cause data theft, service disruption, unauthorized access, and financial losses.

An **Intrusion Detection System (IDS)** is used to monitor network traffic and identify suspicious activities. Conventional IDS approaches generally depend on predefined signatures or rules. Although these methods can detect known attacks effectively, they may have difficulty identifying new or previously unseen attacks.

Deep learning provides an alternative approach by automatically learning useful representations from large datasets. An **autoencoder** is an unsupervised neural network consisting of an encoder and decoder. It learns to reconstruct input data after compressing it into a lower-dimensional representation.

For anomaly detection, the autoencoder can be trained primarily on normal network traffic. When abnormal traffic is provided, the model may produce a larger reconstruction error because the learned representation does not accurately represent the unusual pattern. This reconstruction error can therefore be used to identify network anomalies.

## 1.2 Project Objectives

The main objectives of the project are:

1. To develop an autoencoder-based deep learning framework for network anomaly detection.
2. To preprocess and prepare network traffic data for deep learning.
3. To learn meaningful features using the encoder component of the autoencoder.
4. To identify anomalous network traffic using reconstruction error.
5. To determine an appropriate threshold for distinguishing normal and abnormal traffic.
6. To evaluate the performance of the proposed framework using standard classification metrics.
7. To provide an efficient approach for detecting previously unseen network attacks.

## 1.3 Significance of the Project

The project is significant because cybersecurity systems need to detect both known and unknown threats. An autoencoder-based approach does not require every possible attack pattern to be manually defined.

The system can:

- Learn normal network behavior automatically.
- Detect unusual network traffic.
- Reduce dependence on predefined attack signatures.

- Extract useful features from high-dimensional network data.
- Support early detection of potential intrusions.
- Improve the automation of network security monitoring.

## 1.4 Scope

The scope of this project includes the development of a deep-learning-based anomaly detection system for network traffic.

The project covers:

- Network traffic dataset collection.
- Data preprocessing and normalization.
- Feature preparation.
- Autoencoder model development.
- Model training and validation.
- Reconstruction-error calculation.
- Anomaly threshold determination.
- Normal and anomalous traffic classification.
- Performance evaluation.

The framework can be further extended for real-time network monitoring and deployment in practical cybersecurity environments.

## 1.5 Methodology Overview

The proposed methodology consists of the following stages:

**Dataset Collection → Data Preprocessing → Feature Scaling → Autoencoder Training → Latent Feature Extraction → Data Reconstruction → Reconstruction Error Calculation → Threshold Selection → Anomaly Detection → Performance Evaluation**

The model is trained to reconstruct normal network traffic. During testing, the reconstruction error is calculated for each network record. Records with an error above the selected threshold are classified as anomalous, while records below the threshold are classified as normal.

## **Chapter 2 – Problem Identification And Analysis**

### **2.1 Description of the Problem**

Modern networks generate large volumes of traffic continuously. Among this traffic, identifying malicious or abnormal activities is challenging because attacks can vary significantly in their characteristics.

Traditional intrusion detection methods may fail when:

- The attack is previously unknown.
- Attack patterns change over time.
- Large amounts of network traffic must be analyzed.
- The system encounters complex and high-dimensional data.

Therefore, there is a need for an automated system that can learn network behavior and identify unusual patterns without requiring predefined signatures for every attack.

### **2.2 Evidence of the Problem**

Cyberattacks such as denial-of-service, scanning, brute-force attempts, malware activity, and unauthorized access can produce unusual network patterns.

Examples of potentially suspicious characteristics include:

- Unusually high traffic volume.
- Abnormal connection frequency.
- Unexpected protocol usage.
- Large numbers of failed connections.
- Unusual source or destination behavior.
- Sudden changes in packet-related features.

These characteristics demonstrate the importance of intelligent anomaly detection techniques.

### **2.3 Stakeholders**

The major stakeholders of the proposed system include:

#### **1. Network Administrators**

They can use the system to monitor network activity and identify suspicious traffic.

#### **2. Cybersecurity Professionals**

Security teams can use anomaly detection results to investigate potential threats.

#### **3. Organizations**

Organizations can use the system to improve network security and reduce the impact of cyberattacks.

#### **4. System Administrators**

They can use detected anomalies to identify unusual system or network behavior.

## **5. End Users**

Improved network security helps protect users' data and services from unauthorized activities.

### **2.4 Supporting Data/Research**

The framework can be evaluated using publicly available intrusion-detection datasets such as **NSL-KDD, UNSW-NB15, CIC-IDS2017, or similar network security datasets.**

Typical network traffic features may include:

- Protocol type.
- Source and destination information.
- Connection duration.
- Packet counts.
- Byte counts.
- Service information.
- Connection status.
- Traffic rates.

The dataset is divided into training and testing data. Normal traffic is used for learning the normal representation, while test data containing both normal and anomalous samples is used for evaluating detection performance.

## Chapter 3 – Solution Design and Implementation

### 3.1 Development and Design Process

The development process consists of several stages.

#### Step 1: Dataset Collection

A suitable network intrusion dataset is selected. The dataset contains network traffic records and corresponding information about normal and attack activities.

#### Step 2: Data Preprocessing

The dataset is cleaned by:

- Removing duplicate records.
- Handling missing values.
- Encoding categorical variables.
- Removing irrelevant features.
- Scaling numerical features.

#### Step 3: Model Development

An autoencoder architecture is designed using neural network layers.

#### Step 4: Model Training

The autoencoder is trained to reconstruct normal network traffic.

#### Step 5: Anomaly Detection

Reconstruction error is calculated for test samples.

#### Step 6: Threshold Selection

A threshold is selected to separate normal and anomalous traffic.

#### Step 7: Evaluation

The system is evaluated using accuracy, precision, recall, F1-score, and confusion matrix.

### 3.2 Tools and Technologies Used

| Tool/Technology               | Purpose                               |
|-------------------------------|---------------------------------------|
| Python                        | Main programming language             |
| Google Colab/Jupyter Notebook | Model development and experimentation |
| NumPy                         | Numerical computation                 |
| Pandas                        | Dataset processing                    |
| Matplotlib                    | Data visualization                    |
| Scikit-learn                  | Preprocessing and evaluation          |
| TensorFlow/Keras              | Autoencoder development               |
| Network Intrusion Dataset     | Model training and testing            |

### 3.3 Solution Overview

The proposed framework consists of five major components:

#### Data Input

Network traffic data is provided to the system.

#### Preprocessing

Data is cleaned, encoded, normalized, and converted into a suitable numerical format.

#### Encoder

The encoder compresses the input into a smaller latent representation.

For example:

**Input Layer → Dense Layer → Dense Layer → Bottleneck Layer**

#### Decoder

The decoder reconstructs the original input:

**Bottleneck → Dense Layer → Dense Layer → Output Layer**

#### Anomaly Detection

The original input and reconstructed output are compared. The reconstruction error is calculated.

A commonly used reconstruction error is:

$$MSE = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

where:

- $x_i$  = original input
- $\hat{x}_i$  = reconstructed output
- $n$  = number of features

If:

$$MSE > Threshold$$

the traffic is classified as **anomalous**.

Otherwise, it is classified as **normal**.

### 3.4 Engineering Standards Applied

The project follows general software engineering and cybersecurity principles such as:

- Secure data processing.
- Proper dataset handling.
- Modular software development.
- Model validation and testing.
- Reproducible experimentation.
- Data privacy considerations.

- Secure network monitoring practices.

Cybersecurity practices can also be aligned with established security frameworks and guidelines such as **NIST cybersecurity principles**.

### **3.5 Solution Justification**

An autoencoder is suitable for this project because it can learn patterns without requiring manually designed rules for every possible attack.

The main advantages are:

1. **Unsupervised learning:** The model can learn normal patterns without requiring complete attack labels during training.
2. **Feature learning:** The encoder automatically learns useful representations.
3. **Dimensionality reduction:** The bottleneck layer provides a compressed representation.
4. **Anomaly detection:** Unusual traffic can produce higher reconstruction errors.
5. **Adaptability:** The framework can potentially identify previously unseen patterns.
6. **Automation:** Large volumes of network traffic can be processed automatically.

## Chapter 4 – Results and Recommendations

### 4.1 Evaluation of Results

The performance of the proposed framework can be evaluated using:

#### Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

#### Precision

$$Precision = \frac{TP}{TP + FP}$$

#### Recall

$$Recall = \frac{TP}{TP + FN}$$

#### F1-Score

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

where:

- **TP** = True Positive
- **TN** = True Negative
- **FP** = False Positive
- **FN** = False Negative

The reconstruction-error graph can also be used to observe the separation between normal and anomalous network traffic.

**Note:** Actual numerical performance values should be filled in after running your model on your selected dataset.

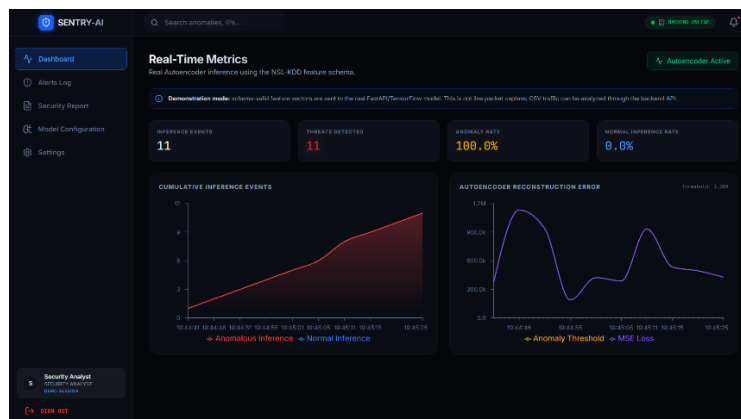


Fig:4.1 DashBoard



| Alert ID          | Timestamp           | Source IP           | Dest IP             | MSE Error    | Severity | Status |
|-------------------|---------------------|---------------------|---------------------|--------------|----------|--------|
| ALT-1788185733271 | 2026-08-31 14:15:23 | NSL-KDD demo stream | NSL-KDD demo stream | 153536.4349  | CRITICAL | New    |
| ALT-1788185729986 | 2026-08-31 14:15:28 | NSL-KDD demo stream | NSL-KDD demo stream | 248618.3474  | CRITICAL | New    |
| ALT-1788185729977 | 2026-08-31 14:15:28 | NSL-KDD demo stream | NSL-KDD demo stream | 719661.2485  | CRITICAL | New    |
| ALT-1788185723183 | 2026-08-31 14:15:23 | NSL-KDD demo stream | NSL-KDD demo stream | 427876.1181  | CRITICAL | New    |
| ALT-1788185718271 | 2026-08-31 14:15:18 | NSL-KDD demo stream | NSL-KDD demo stream | 494537.7483  | CRITICAL | New    |
| ALT-1788185731183 | 2026-08-31 14:15:13 | NSL-KDD demo stream | NSL-KDD demo stream | 533768.3118  | CRITICAL | New    |
| ALT-1788185788274 | 2026-08-31 14:15:08 | NSL-KDD demo stream | NSL-KDD demo stream | 1884868.5183 | CRITICAL | New    |
| ALT-1788185788273 | 2026-08-31 14:15:08 | NSL-KDD demo stream | NSL-KDD demo stream | 934871.8138  | CRITICAL | New    |
| ALT-1788185788173 | 2026-08-31 14:15:03 | NSL-KDD demo stream | NSL-KDD demo stream | 387466.8216  | CRITICAL | New    |
| ALT-1788185698271 | 2026-08-31 14:14:58 | NSL-KDD demo stream | NSL-KDD demo stream | 422293.3813  | CRITICAL | New    |
| ALT-1788185691188 | 2026-08-31 14:14:53 | NSL-KDD demo stream | NSL-KDD demo stream | 188849.2357  | CRITICAL | New    |

Fig:4.2 Alert Log

| Time                | Source              | Destination         | Protocol | MSE         | Severity |
|---------------------|---------------------|---------------------|----------|-------------|----------|
| 2026-08-31 14:15:23 | NSL-KDD demo stream | NSL-KDD demo stream | UDP      | 153536.4349 | Critical |
| 2026-08-31 14:15:28 | NSL-KDD demo stream | NSL-KDD demo stream | UDP      | 248618.3474 | Critical |
| 2026-08-31 14:15:28 | NSL-KDD demo stream | NSL-KDD demo stream | ICMP     | 719661.2485 | Critical |

Fig: 4.3 Data Report

## 4.2 Challenges Encountered

Several challenges may occur during the development of the system:

- Large dataset size.
- Imbalanced normal and attack samples.
- Categorical network features.
- Missing or duplicate values.
- Selection of suitable model architecture.
- Determining an appropriate anomaly threshold.
- Avoiding overfitting.
- Computational requirements during training.

These challenges can be addressed through preprocessing, normalization, appropriate model configuration, validation, and careful threshold selection.

### **4.3 Possible Improvements**

The system can be improved by:

1. Using larger and more diverse datasets.
2. Optimizing the autoencoder architecture.
3. Applying different types of autoencoders.
4. Performing systematic hyperparameter tuning.
5. Improving threshold-selection techniques.
6. Combining autoencoders with other detection approaches.
7. Implementing real-time network traffic monitoring.
8. Using continuous model updating for changing network behavior.

### **4.4 Recommendations**

The following recommendations can improve practical deployment:

- Regularly retrain the model using updated network data.
- Monitor false-positive and false-negative rates.
- Use multiple evaluation metrics rather than accuracy alone.
- Protect the training dataset from manipulation.
- Integrate the system with existing security monitoring tools.
- Provide alerts when high-confidence anomalies are detected.
- Periodically evaluate model performance against new attack patterns.

## Chapter 5 – Reflection on Learning and Personal Development

### 5.1 Key Learning Outcomes

Through this project, the following skills and knowledge were developed:

- Understanding of network intrusion detection.
- Knowledge of deep learning concepts.
- Understanding of autoencoder architecture.
- Experience with Python programming.
- Dataset preprocessing skills.
- Understanding of model training and validation.
- Knowledge of anomaly detection.
- Experience with performance evaluation.
- Data visualization and interpretation skills.

### 5.2 Challenges Encountered and Overcome

During the project, challenges were encountered in understanding network datasets, preprocessing categorical and numerical features, designing the autoencoder, and selecting the anomaly threshold. These challenges were overcome through experimentation, studying deep learning concepts, testing different preprocessing techniques, and evaluating the model using different performance measures. The project improved problem-solving ability and provided practical experience in developing a machine-learning-based cybersecurity application.

### 5.3 Application of Engineering Standards

Engineering principles were applied throughout the development process.

The project followed a structured development methodology consisting of:

**Requirement Analysis → Design → Implementation → Testing → Evaluation → Improvement**

Attention was given to reliability, security, maintainability, data handling, and systematic testing.

### 5.4 Insights into the Industry

The project provided an understanding of how artificial intelligence and cybersecurity are increasingly being combined to protect modern networks.

Modern organizations need automated security systems because the amount of network traffic is continuously increasing. Machine learning and deep learning can help security teams identify suspicious patterns and prioritize potential threats.

The project also demonstrated the importance of continuous monitoring because network behavior and attack techniques can change over time.

## **5.5 Conclusion of Personal Development**

The project improved both technical and analytical skills. It provided practical knowledge of Python, deep learning, network security, data preprocessing, model evaluation, and anomaly detection.

The experience also improved problem-solving, research, documentation, and project-development skills.

## Chapter 6 – Conclusion

The Autoencoder-Based Deep Learning Framework for Network Intrusion and Anomaly Detection provides an effective approach for identifying unusual and potentially malicious activities in modern network environments. As cyber threats continue to become more sophisticated, traditional security techniques based only on predefined rules and known attack signatures may not be sufficient to detect new or previously unseen attacks. The proposed framework addresses this limitation by using an autoencoder to learn the normal patterns and characteristics of network traffic and identify deviations from those patterns.

The autoencoder consists mainly of an encoder, bottleneck layer, and decoder. The encoder compresses the input network data into a lower-dimensional representation, while the decoder reconstructs the original input from this representation. During training, the model learns the important features of normal network behaviour. When abnormal or intrusive traffic is provided, the reconstruction becomes less accurate, resulting in a higher reconstruction error. This error can be used as an anomaly score to distinguish normal traffic from suspicious network activity.

The framework can support important stages such as data preprocessing, feature extraction, dimensionality reduction, model training, anomaly detection, and performance evaluation. Proper preprocessing and normalization of network traffic data improve the learning capability of the model. The bottleneck representation also helps reduce unnecessary information while preserving important patterns in the data. By setting an appropriate threshold for reconstruction error, the system can classify network activities as normal or anomalous.

## Chapter 7 – References

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
2. Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504–507.
3. Sakurada, M., & Yairi, T. (2014). Anomaly detection using autoencoders with nonlinear dimensionality reduction. *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, 4–11.
4. Shone, N., Ngoc, T. N., Phai, V. D., & Shi, Q. (2018). A deep learning approach to network intrusion detection. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2(1), 41–50.
5. Mirsky, Y., Doitshman, T., Elovici, Y., & Shabtai, A. (2018). Kitsune: An ensemble of autoencoders for online network intrusion detection. *Proceedings of the Network and Distributed System Security Symposium (NDSS)*.
6. Ring, M., Wunderlich, S., Grödl, D., Landes, D., & Hotho, A. (2019). A survey of network-based intrusion detection data sets. *Computers & Security*, 86, 147–167.
7. Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A., & Venkatraman, S. (2019). Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7, 41525–41550.
8. Ferrag, M. A., Maglaras, L., Moschoyiannis, S., & Janicke, H. (2020). Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *Journal of Information Security and Applications*, 50, 102419.
9. Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). A deep learning approach for network intrusion detection system. *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies*, 21–26.
10. Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 1–6.

## Chapter 8 – Appendices

### 8.1 User Manual

#### Step 1

Open the Python/Google Colab environment.

#### Step 2

Install or import the required libraries.

#### Step 3

Load the network intrusion dataset.

#### Step 4

Perform preprocessing and feature scaling.

#### Step 5

Create the autoencoder model.

#### Step 6

Train the model using the prepared normal network data.

#### Step 7

Calculate reconstruction errors.

#### Step 8

Apply the selected threshold.

#### Step 9

Classify network records as:

- **Normal**
- **Anomalous/Intrusion**

#### Step 10

View the performance metrics and graphs.

### 8.2 Code Snippets

The appendix can contain important sections of your Python implementation, such as:

- Dataset loading.
- Data preprocessing.
- Feature scaling.
- Autoencoder architecture.
- Model training.
- Reconstruction-error calculation.
- Threshold calculation.

- Anomaly classification.
- Confusion matrix.
- Performance evaluation.

### **8.3 Raw Dataset**

The raw dataset used for developing and evaluating the proposed intrusion detection framework can be included in this section or provided as a separate dataset file.

The dataset should contain relevant network traffic features and, where available, labels indicating normal or attack traffic.